



Последовательности: списки и кортежи

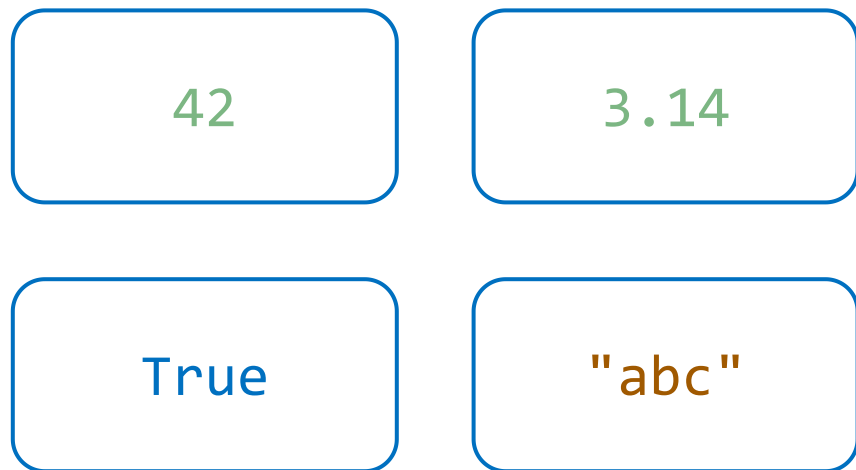
Докладчик: Евграфов Михаил



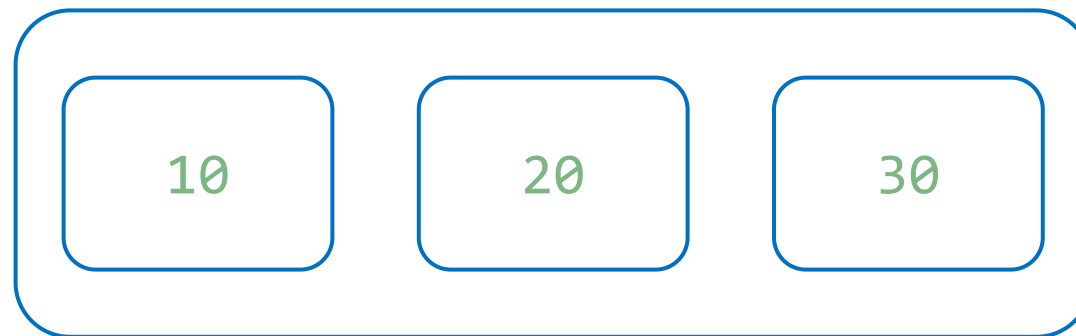
Список: обзор

Составной тип данных

скалярные: одно значение



составной: контейнер



хранит ссылки на другие объекты

Список — контейнер

```
empty_list = []
```

```
numbers = [10, 20, 30]
```

```
print(numbers) # [10, 20, 30]
```

Характеристики списка

гетерогенный

ограниченный

упорядоченный

изменяемый

итерируемый

Гетерогенность

```
mixed_values = [  
    42, "hello", 3.14, True, None, [1, 2, 3]  
]
```

```
print(mixed_values)  
# [42, 'hello', 3.14, True, None, [1, 2, 3]]
```

Однородные списки

```
ages = [18, 25, 42]
```

```
names = ["Alice", "Bob", "Charlie"]
```

```
print(ages)    # [18, 25, 42]
```

```
print(names)   # ['Alice', 'Bob', 'Charlie']
```

Ограниченность

```
fruits = ["apple", "banana", "cherry"]
```

```
print(len(fruits))    # 3
```

```
print(len([]))        # 0
```

Упорядоченность

```
ordered_colors = ["red", "green", "blue"]
```

```
print(ordered_colors[0])
```

```
# red - первый элемент
```

```
print(ordered_colors[2])
```

```
# blue - третий элемент
```

Изменяемость

```
scores = [10, 20, 30]
identity_before = id(scores)

scores.append(40)
scores[0] = 100

print(scores)
# [100, 20, 30, 40]
print(id(scores) == identity_before)
# True
```

Итерируемость

```
fruits = ["apple", "banana", "cherry"]
```

```
for fruit in fruits:  
    print(fruit)
```

```
# apple
```

```
# banana
```

```
# cherry
```

Где применяются списки

**наборы однородных
данных**

**накопление
результатов**

очереди и стеки

таблицы и матрицы

**промежуточные
данные**



Создание списков

Списковый литерал

```
empty_list = []  
single_item_list = [42]  
several_items_list = [1, 2, 3, 4, 5]  
mixed_list = [1, "two", 3.0, True]
```

Вложенные списки

```
matrix = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9],  
]
```

list()

```
list("abc")  
# ['a', 'b', 'c']  
list((1, 2, 3))  
# [1, 2, 3]  
list(range(5))  
# [0, 1, 2, 3, 4]  
list()  
# []
```

list() создает копию

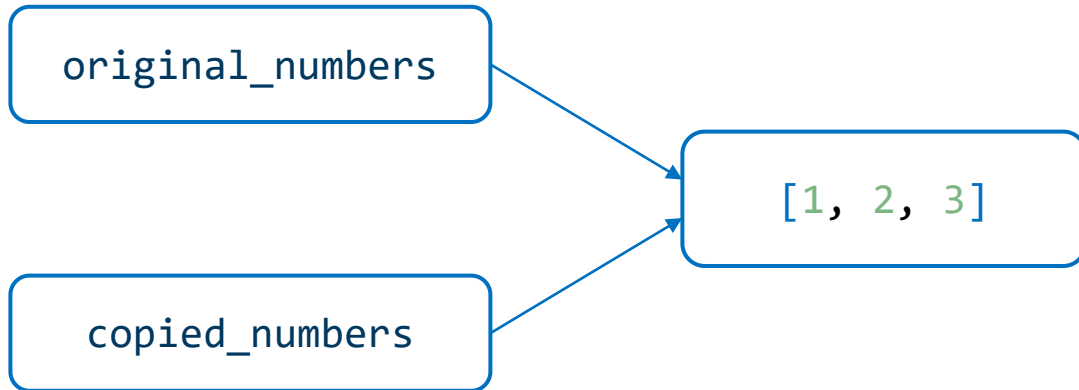
```
original_numbers = [1, 2, 3]  
copied_numbers = list(original_numbers)
```

```
# добавляем элемент в конец списка  
copied_numbers.append(4)
```

```
print(original_numbers)  
# [1, 2, 3]  
print(copied_numbers)  
# [1, 2, 3, 4]
```

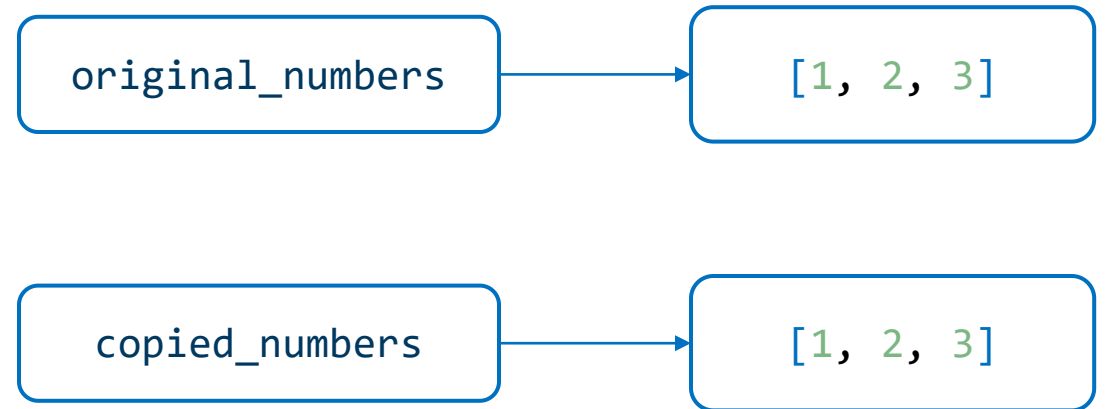
Копия и второе имя

```
copied_numbers = original_numbers
```



один объект, два имени

```
copied_numbers = list(original_numbers)
```



два независимых объекта

Списковое включение

[выражение **for** переменная **in** источник]

```
squares = [number ** 2 for number in range(1, 6)]
```

```
print(squares)  
# [1, 4, 9, 16, 25]
```

Обычное включение

```
words = ["python", "list", "tuple"]  
  
word_lengths = [len(word) for word in words]  
  
print(word_lengths)  
# [6, 4, 5]
```

Включение с if

[выражение **for** переменная **in** источник **if** условие]

```
numbers = range(1, 11)
```

```
even_numbers = [  
    number for number in numbers if number % 2 == 0  
]
```

```
print(even_numbers)  
# [2, 4, 6, 8, 10]
```

Тот же результат циклом

```
even_numbers = []
```

```
for number in numbers:  
    if number % 2 == 0:  
        even_numbers.append(number)
```

```
print(even_numbers)  
# [2, 4, 6, 8, 10]
```

Вложенные циклы

[выражение **for** переменная_1 in источник_1 **for** переменная_2 in источник_2]

```
coordinate_pairs = [  
    (row, column) for row in range(3) for column in range(2)  
]
```

```
print(coordinate_pairs)  
# [(0, 0), (0, 1), (1, 0), (1, 1), (2, 0), (2, 1)]
```

Тот же результат циклом

```
coordinate_pairs = []

for row in range(3):
    for column in range(2):
        coordinate_pairs.append((row, column))

print(coordinate_pairs)
# [(0, 0), (0, 1), (1, 0), (1, 1), (2, 0), (2, 1)]
```

Практический пример

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

```
flat_values = [  
    value for row in matrix for value in row  
]
```

```
print(flat_values)
```

```
# [1, 2, 3, 4, 5, 6, 7, 8, 9]
```



Длина и индексы

len()

```
fruits = ["apple", "banana", "cherry"]
```

```
print(len(fruits))    # 3
```

```
print(len([]))        # 0
```

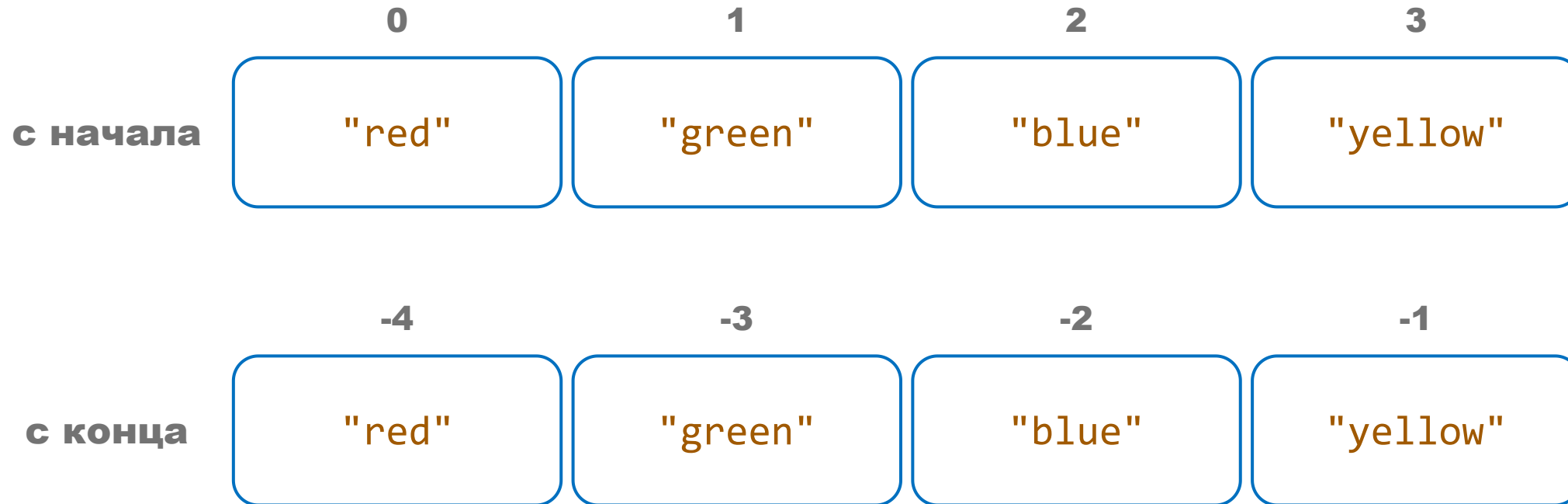
len() и вложенность

```
nested_list = [1, 2, [3, 4, 5]]
```

```
print(len(nested_list))
```

```
# 3
```

Индексы списка



Положительные индексы

```
colors = ["red", "green", "blue", "yellow"]
```

```
print(colors[0])
```

```
# red
```

```
print(colors[1])
```

```
# green
```

```
print(colors[3])
```

```
# yellow
```

Отрицательные индексы

```
colors = ["red", "green", "blue", "yellow"]
```

```
print(colors[-1])
```

```
# yellow
```

```
print(colors[-2])
```

```
# blue
```

```
print(colors[-4])
```

```
# red
```

IndexError

```
colors = ["red", "green", "blue"]
```

```
print(colors[3])
```

```
# IndexError: list index out of range
```

```
print(colors[-4])
```

```
# IndexError: list index out of range
```

Синтаксис среза

```
list[start:stop:step]
```

start

включительно
по умолчанию — начало

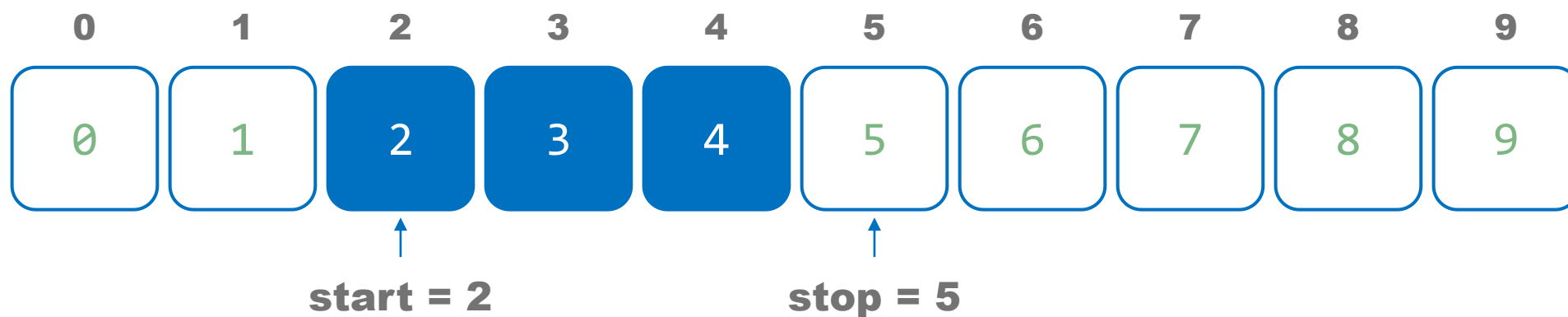
stop

не входит в результат
по умолчанию — конец

step

шаг
по умолчанию — 1

Срез numbers[2:5]



результат — новый список [2, 3, 4]

Срезы

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
print(numbers[2:5])    # [2, 3, 4]
print(numbers[:4])     # [0, 1, 2, 3]
print(numbers[6:])     # [6, 7, 8, 9]
print(numbers[:])      # [0, 1, ..., 9]
print(numbers[::])     # [0, 1, ..., 9]
print(numbers[::2])    # [0, 2, 4, 6, 8]
print(numbers[::-1])   # [9, 8, ..., 0]
```

Отрицательные срезы

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
print(numbers[-3:])  
# [7, 8, 9]
```

```
print(numbers[:-3])  
# [0, 1, ..., 6]
```

Срез без IndexError

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
print(numbers[5:100])  
# [5, 6, 7, 8, 9]
```

```
print(numbers[100:])  
# []
```

Срез — всегда список

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
print(numbers[2:3])
```

```
# [2] - список из одного элемента
```

```
print(numbers[2])
```

```
# 2 - а вот это уже сам элемент
```

Объект slice

```
explicit_slice = slice(1, 8, 2)
```

```
print(type(explicit_slice))      # <class 'slice'>
print(numbers[explicit_slice])  # [1, 3, 5, 7]
print(numbers[1:8:2])           # [1, 3, 5, 7]
```

```
reverse_slice = slice(None, None, -1)
```

```
print(numbers[reverse_slice])
# [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

Практический пример

```
DATE_SLICE = slice(0, 10)
LEVEL_SLICE = slice(11, 16)
MESSAGE_SLICE = slice(17, None)

log_lines = [
    "2024-05-17 ERROR disk is full",
    "2024-05-18 INFO backup finished",
]

for log_line in log_lines:
    print(log_line[LEVEL_SLICE], log_line[DATE_SLICE], log_line[MESSAGE_SLICE])
# ERROR 2024-05-17 disk is full
# INFO 2024-05-18 backup finished
```



Изменение списков

Замена по индексу

```
fruits = ["apple", "banana", "cherry"]  
fruits[1] = "blueberry"
```

```
print(fruits)  
# ['apple', 'blueberry', 'cherry']
```

Замена среза

```
numbers = [0, 1, 2, 3, 4, 5]  
numbers[1:4] = [10, 20]  
  
print(numbers)  
# [0, 10, 20, 4, 5]
```

Вставка через срез

```
letters = ["a", "b", "c"]  
letters[1:1] = ["x", "y"]  
  
print(letters)  
# ['a', 'x', 'y', 'b', 'c']
```

Удаление через срез

```
numbers = [0, 1, 2, 3, 4, 5]
```

```
numbers[1:4] = []
```

```
print(numbers)
```

```
# [0, 4, 5]
```

Методы списка

ИЗМЕНЯЮТ СПИСОК «НА МЕСТЕ»

`append()`

`insert()`

`extend()`

`remove()`

`pop()`

`clear()`

`sort()`

`reverse()`

ТОЛЬКО ЧИТАЮТ ИНФОРМАЦИЮ

`index()`

`count()`

`copy()`

Методы вставки

```
groceries = ["milk", "bread"]
```

```
groceries.append("eggs")  
print(groceries)  
# ['milk', 'bread', 'eggs']
```

```
groceries.insert(1, "butter")  
print(groceries)  
# ['milk', 'butter', 'bread', 'eggs']
```

```
groceries.extend(["tea", "coffee"])  
print(groceries)  
# ['milk', 'butter', 'bread', 'eggs', 'tea', 'coffee']
```

append() vs extend()

```
numbers = [1, 2, 3]
numbers.append([4, 5])
print(numbers)
# [1, 2, 3, [4, 5]]
```

```
another_numbers = [1, 2, 3]
another_numbers.extend([4, 5])
print(another_numbers)
# [1, 2, 3, 4, 5]
```

Методы удаления

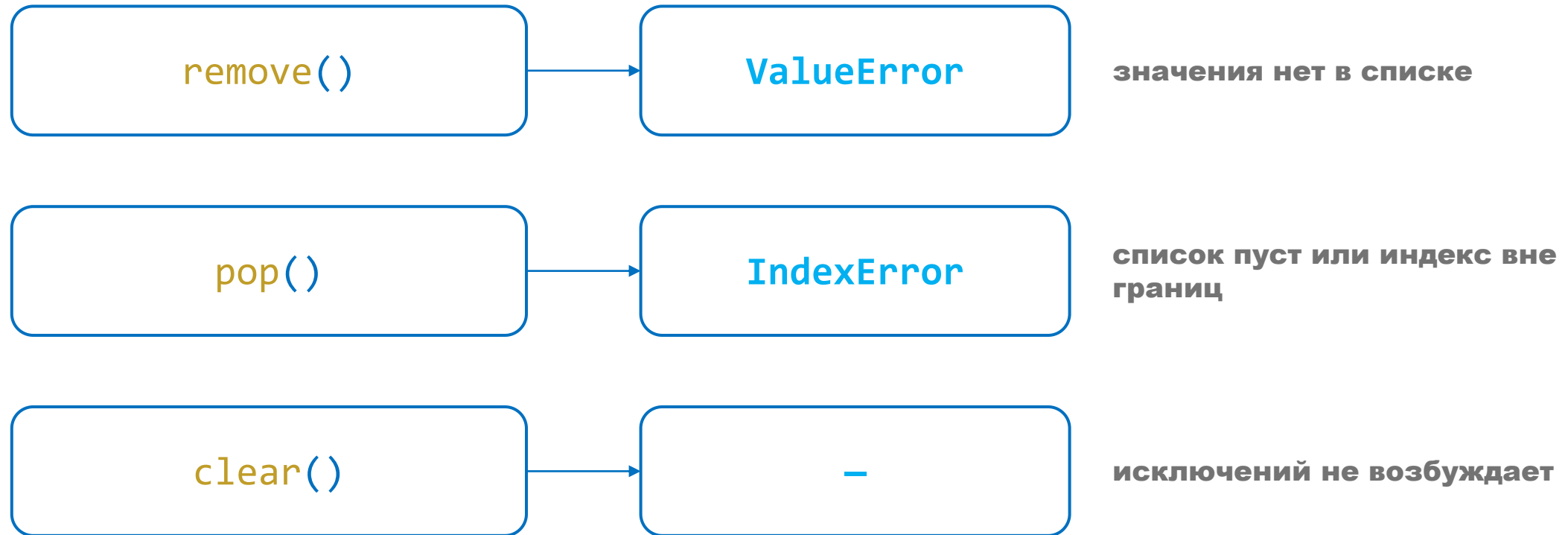
```
groceries.remove("butter")  
print(groceries)  # ['milk', 'bread', 'eggs', 'tea', 'coffee']
```

```
last_item = groceries.pop()  
print(last_item)  # coffee  
print(groceries)  # ['milk', 'bread', 'eggs', 'tea']
```

```
first_item = groceries.pop(0)  
print(first_item)  # milk  
print(groceries)  # ['bread', 'eggs', 'tea']
```

```
groceries.clear()  
print(groceries)  # []
```

Исключения при удалении



Изменение структуры

```
groceries = ["bread", "eggs", "tea"]
```

```
groceries.sort()  
print(groceries)  
# ['bread', 'eggs', 'tea']
```

```
groceries.reverse()  
print(groceries)  
# ['tea', 'eggs', 'bread']
```

index(), count(), copy()

```
numbers = [3, 1, 4, 1, 5]
```

```
print(numbers.index(4))
```

```
# 2
```

```
print(numbers.count(1))
```

```
# 2
```

```
print(numbers.copy())
```

```
# [3, 1, 4, 1, 5]
```

del

```
numbers = [10, 20, 30, 40, 50]
```

```
del numbers[0]  
print(numbers)  
# [20, 30, 40, 50]
```

```
del numbers[1:3]  
print(numbers)  
# [20, 50]
```

del

```
values = [1, 2, 3]
del values[:]
print(values)
# []
```

```
del values
print(values)
# NameError: name 'values' is not defined
```



Итерируемость

Цикл for

```
fruits = ["apple", "banana", "cherry"]
```

```
for fruit in fruits:  
    print(fruit)
```

```
# apple
```

```
# banana
```

```
# cherry
```

Следствие итерируемости

```
scores = [85, 92, 78, 90]
```

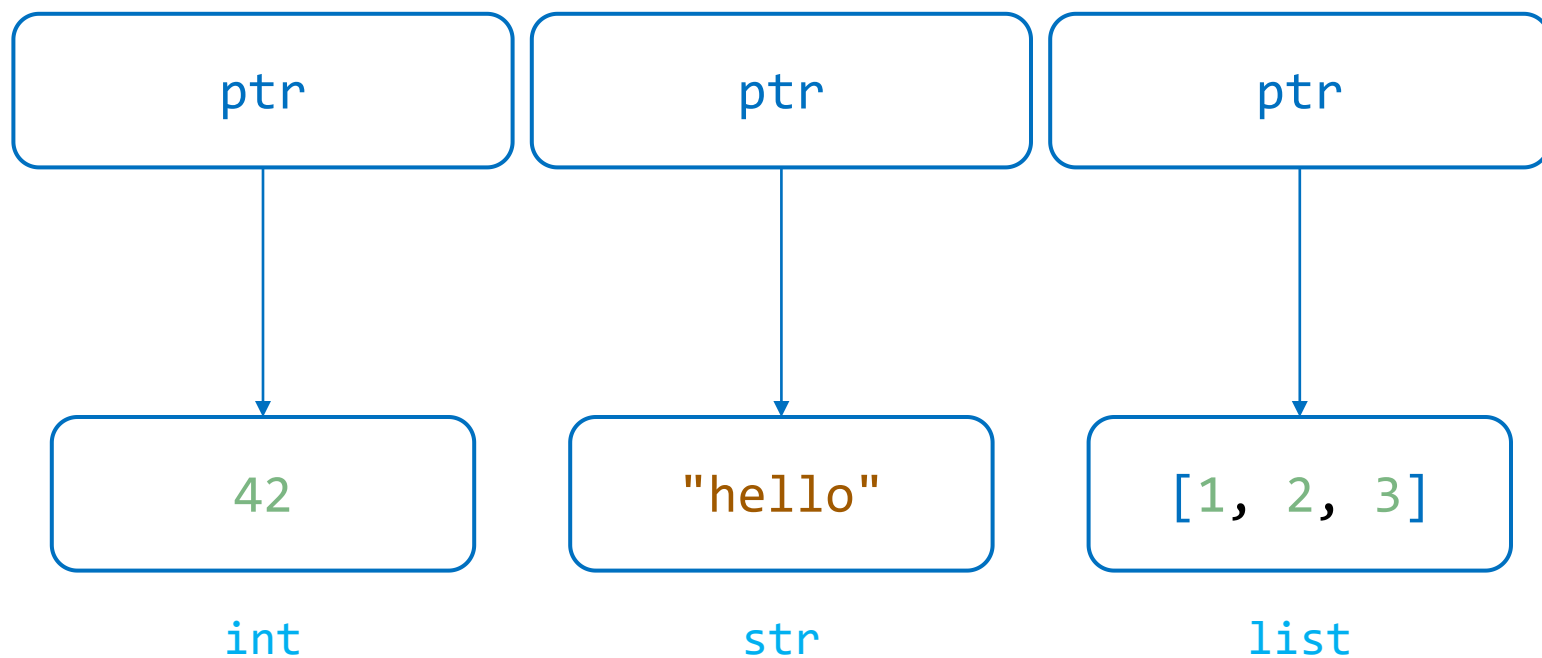
```
print(sum(scores))           # 345
print(max(scores))           # 92
print(min(scores))           # 78
print(sorted(scores))        # [78, 85, 90, 92]
print(list(reversed(scores))) # [90, 78, 92, 85]
```



Списки в CPython

Массив указателей

`list` — массив указателей



**указатели
одного размера**

**объекты живут
отдельно**

Запас памяти

есть запас



`append()` — быстро

запас закончился



новый, больший массив





Кортеж: обзор

Кортеж — контейнер

```
empty_tuple = ()  
coordinates = (10, 20)
```

```
print(coordinates)  
# (10, 20)
```

list и tuple

list

гетерогенный

ограниченный

упорядоченный

итерируемый

изменяем

tuple

гетерогенный

ограниченный

упорядоченный

итерируемый

неизменяем

Гетерогенность

```
person_record = ("Alice", 30, True)
```

```
print(person_record)  
# ('Alice', 30, True)
```

Ограниченность

```
print(len(person_record))    # 3  
print(len(()))              # 0
```

Упорядоченность

```
rgb_color = (255, 0, 0)
```

```
print(rgb_color[0])  
# 255
```

Неизменяемость

```
rgb_color = (255, 0, 0)
```

```
rgb_color[0] = 100
```

```
# TypeError: 'tuple' object does not support item assignment
```

Итерируемость

```
for value in (1, 2, 3):  
    print(value)
```

```
# 1
```

```
# 2
```

```
# 3
```

Где применяются кортежи

**записи
фиксированной
структуры**

**возврат нескольких
значений**

**защита от
случайных
изменений**



Создание кортежей

Литералы

```
coordinates = (10, 20)
coordinates_without_parentheses = 10, 20

print(coordinates)
# (10, 20)
print(coordinates_without_parentheses)
# (10, 20)
print(coordinates == coordinates_without_parentheses)
# True
```

Кортеж из одного элемента

```
not_a_tuple = (42)
single_item_tuple = (42,)
```

```
print(type(not_a_tuple))      # <class 'int'>
print(type(single_item_tuple)) # <class 'tuple'>
print(single_item_tuple)      # (42,)
```

Пустой кортеж

```
empty_tuple = ()
```

```
print(type(empty_tuple))  
# <class 'tuple'>
```

tuple()

```
tuple("abc")  
# ('a', 'b', 'c')  
tuple([1, 2, 3])  
# (1, 2, 3)  
tuple(range(5))  
# (0, 1, 2, 3, 4)  
tuple()  
# ()
```

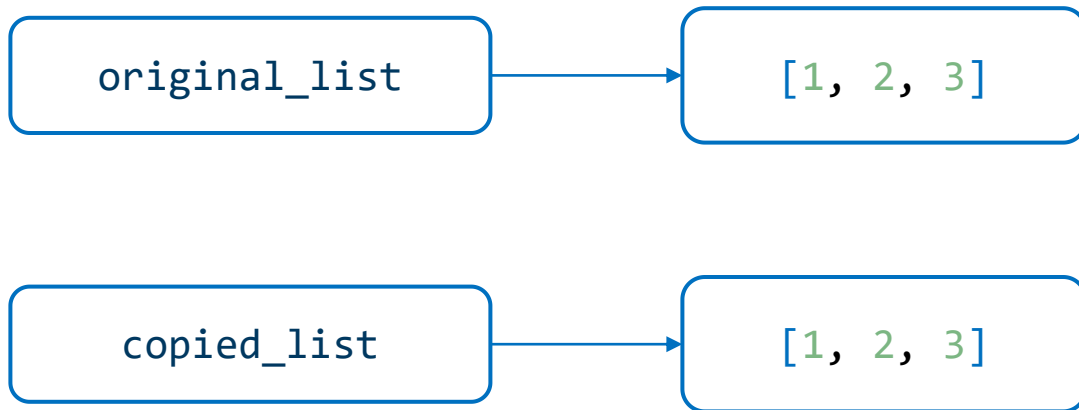
tuple() от кортежа

```
original_tuple = (1, 2, 3)
same_tuple = tuple(original_tuple)

print(same_tuple is original_tuple)
# True
```

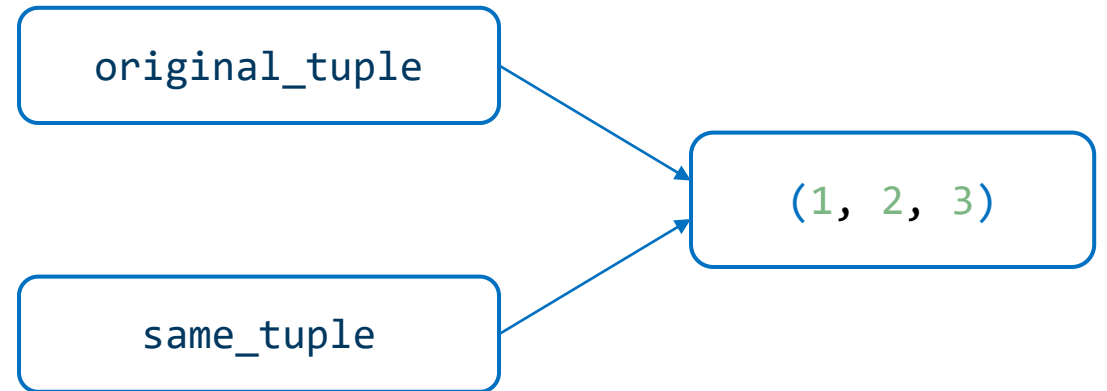
list() vs tuple()

```
copied_list = list(original_list)
```



list(): новый объект

```
same_tuple = tuple(original_tuple)
```



tuple(): тот же объект

Кортежные включения

```
squares_generator = (number ** 2 for number in range(1, 6))  
squares_tuple = tuple(number ** 2 for number in range(1, 6))  
  
print(type(squares_generator))    # <class 'generator'>  
print(squares_tuple)              # (1, 4, 9, 16, 25)
```



Неизменяемость

Чтение по индексу

```
rgb_color = (255, 0, 0)
```

```
print(rgb_color[0])    # 255
```

```
print(rgb_color[-1])   # 0
```

```
print(rgb_color[5])
```

```
# IndexError: tuple index out of range
```

Срезы кортежа

```
numbers = (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)
```

```
print(numbers[2:5])
```

```
# (2, 3, 4)
```

```
print(numbers[:4])
```

```
# (0, 1, 2, 3)
```

```
print(numbers[::2])
```

```
# (0, 2, 4, 6, 8)
```

```
print(numbers[::-1])
```

```
# (9, 8, 7, 6, 5, 4, 3, 2, 1, 0)
```

Запись невозможна

```
rgb_color = (255, 0, 0)
```

```
rgb_color[0] = 100
```

```
# TypeError: 'tuple' object does not support item assignment
```

Изменяемое внутри

```
record = ("Alice", [85, 90, 78])
```

```
record[1].append(100)
```

```
print(record)
```

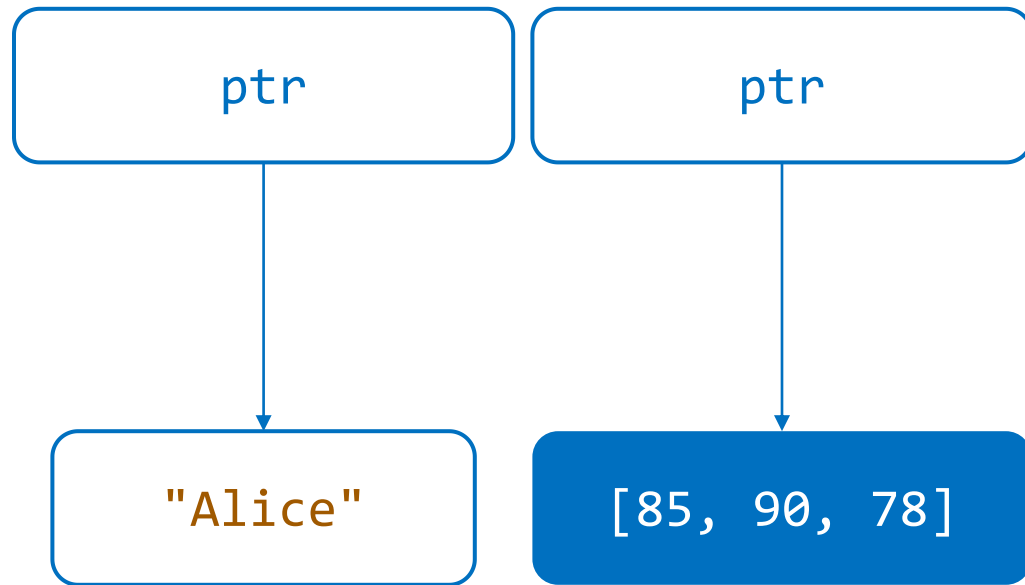
```
# ('Alice', [85, 90, 78, 100])
```

```
record[1] = [0, 0, 0]
```

```
# TypeError: 'tuple' object does not support item assignment
```

Ссылки в кортеже

tuple



ссылки менять нельзя

**содержимое списка
менять можно**



Общие операции

Последовательности

sequence: упорядоченные, ограниченные, итерируемые

list

изменяемая

tuple

неизменяемая

str

неизменяемая

Общие операции

`len()`

`in`

`not in`

`+`

`*`

`[index]`

`[start:stop]`

`for`

in & not in

```
fruits_list = ["apple", "banana", "cherry"]  
fruits_tuple = ("apple", "banana", "cherry")
```

```
print("banana" in fruits_list)           # True  
print("mango" in fruits_list)           # False  
print("mango" not in fruits_tuple)      # True
```

Конкатенация +

```
first_list = [1, 2, 3]
second_list = [4, 5]
print(first_list + second_list)
# [1, 2, 3, 4, 5]
```

```
first_tuple = (1, 2, 3)
second_tuple = (4, 5)
print(first_tuple + second_tuple)
# (1, 2, 3, 4, 5)
```

Оператор +=

```
numbers_list = [1, 2, 3]
list_id_before = id(numbers_list)
numbers_list += [4, 5]
```

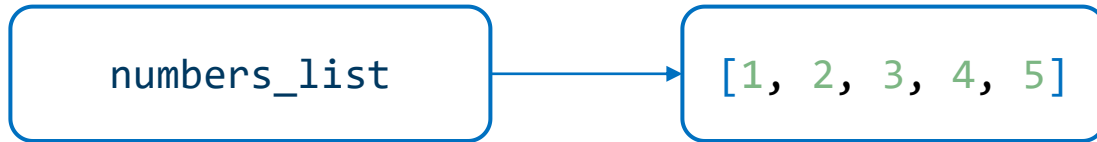
```
print(numbers_list)                # [1, 2, 3, 4, 5]
print(id(numbers_list) == list_id_before) # True
```

```
numbers_tuple = (1, 2, 3)
tuple_id_before = id(numbers_tuple)
numbers_tuple += (4, 5)
```

```
print(numbers_tuple)                # (1, 2, 3, 4, 5)
print(id(numbers_tuple) == tuple_id_before) # False
```

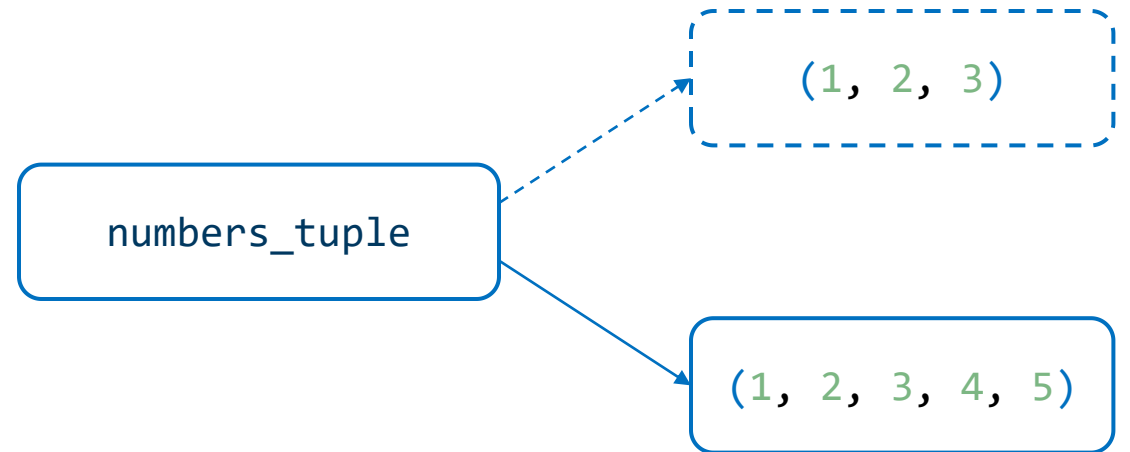
list vs tuple

```
numbers_list += [4, 5]
```



тот же объект

```
numbers_tuple += (4, 5)
```



новый объект

Повторение *

```
print([0] * 4)      # [0, 0, 0, 0]
print((1, 2) * 3)    # (1, 2, 1, 2, 1, 2)
```

```
print([0] * 2.5)
# TypeError: can't multiply sequence by non-int of type 'float'
```

Повторение ≤ 0 раз

```
print([1, 2] * 0)  
# []  
print((1, 2) * -3)  
# ()
```

Оператор `*=`

```
numbers_list = [1, 2]  
numbers_list *= 3  
print(numbers_list)  # [1, 2, 1, 2, 1, 2]
```

```
numbers_tuple = (1, 2)  
numbers_tuple *= 3  
print(numbers_tuple)  # (1, 2, 1, 2, 1, 2)
```



Спасибо за внимание!